

“Stop Trying to Make Fetch Happen”: Towards a Modality-Aware Understanding of Unnecessary Resources on the Web

Rumaisa Habib
Stanford University
rumaisa@cs.stanford.edu

Balaji Balachandran
Stanford University
balajib@cs.stanford.edu

Nurullah Demir
Stanford University
nurullah@cs.stanford.edu

Zakir Durumeric
Stanford University
zakir@cs.stanford.edu

Abstract

Modern websites load hundreds of resources, including images, JavaScript, stylesheets, and fonts. While these web resources enable rich interactive experiences, they also carry heavy network, memory, and CPU costs. Many use cases—like content archival and programmatic automation—do not need the full intended functionality of the website to be successful. Yet these unnecessary resources are fetched regardless, and at scale, this over-fetching compounds into a substantial environmental footprint. However, it is unclear which resources are necessary for which use cases. In this paper, we provide the first modality-aware measurement of unnecessary resources on the web. To do so, we introduce four dimensions of webpage fidelity to characterize modalities on the web. We show that we can reduce 17–98% of network bytes, 39–76% of CPU time, and 11–53% of memory usage while maintaining the relevant content for a given modality. We estimate that modality-aware selective loading could avoid up to 159 million metric tons of CO₂e annually. These findings uncover additional web bloat beyond what was previously understood, and opportunities to build more efficient and sustainable tooling for crawling, interacting with, and archiving content on the web.

CCS Concepts

• **Information systems** → **Web interfaces**; • **Networks** → *Network measurement*.

Keywords

Web Measurement, Web Performance, Web Sustainability, Internet Measurement

ACM Reference Format:

Rumaisa Habib, Balaji Balachandran, Nurullah Demir, and Zakir Durumeric. 2026. “Stop Trying to Make Fetch Happen”: Towards a Modality-Aware Understanding of Unnecessary Resources on the Web. In *Proceedings of the 2026 ACM Internet Measurement Conference (IMC '26)*, October 12–16, 2026, Karlsruhe, Germany. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3777912.3839788>

1 Introduction

Web browsers fetch and execute a large dependency graph of HTML, CSS, and JavaScript resources to render web pages [13, 70]. However, many use cases on the web can succeed with a small subset of those resources. For example, archival tasks (e.g., Wayback Machine [43])

require only specific visible site content yet still incur the CPU, memory, and network costs required for full site interactivity. For such large-scale operations, small inefficiencies compound into substantial aggregate network, computational, and environmental costs [8, 78]. This overhead also affects human visitors with limited data plans or memory-constrained devices [1, 34]. Yet, because web delivery models treat visits uniformly, the full extent of over-fetching remains underexplored.

In this paper, we conduct the first use case-aware measurement of unnecessary resources on the web and quantify the trade-off between system resource utilization and webpage utility. We start by defining four modalities for interacting with the web: text extraction, content archival, programmatic automation, and human browsing. Whether a web resource is required for a page load depends on which aspects of the webpage must be preserved for the given modality. We further define four quantifiable measures of page *fidelity*—textual, structural, visual, and interactive—that measure how well a page rendering matches the original within the context of the modality. For example, programmatic automation through a headless browser requires high-fidelity text, structure, and interactivity, but can compromise on visual fidelity.

We analyze 4,000 websites sampled from the CrUX Top 1 Million Websites. We first consider a strict baseline where web resources are considered unnecessary only if they can be removed while preserving all four dimensions of fidelity. This captures resources that are unnecessary regardless of use case and provides a conservative estimate of unnecessary resources. This restrictive benchmark reveals that 42% of webpages can load fewer resources without any noticeable impact, surfacing a possible median reduction of 14% in page size, 45% in CPU time, and 8% in memory usage. However, we reveal significantly more potential resource savings when we consider access modality. The largest savings are possible when preserving only textual fidelity, but significant savings are also possible for human browsing, where visual and interactive behavior is also preserved. Across our four modalities, we show that we can reduce 17–98% of network bytes, 39–76% of CPU time, and 11–53% of memory usage while maintaining the relevant content.

For many webpages, we can maintain the relevant fidelity without fetching any external resources. When JavaScript is necessary, only a subset of scripts are necessary for some modalities. The primary constraint on potential system resource reductions is visual fidelity, which is substantially lower than other fidelity metrics without external resources. We also find that the potential for resource reduction varies across the web, but not along a single axis. For most modalities, the popularity rank of a website usually does not determine how much reduction is possible. The exception is that more popular sites are more reducible for content archival. Finally, our correlation and predictability analyses show that textual,



This work is licensed under a Creative Commons Attribution 4.0 International License. *IMC '26, Karlsruhe, Germany*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2327-8/2026/10
<https://doi.org/10.1145/3777912.3839788>

structural, visual, and interactive fidelity have some shared structure but are not redundant, so measuring one or more dimensions cannot reliably predict fidelity along the others.

Our results show that a single definition of resource necessity underestimates both the scale and structure of unnecessary resources on the web. We hope that these findings can inform the design of future systems that access or measure the web. Our analysis code and data are available at: <https://github.com/stanford-esrg/fetch-2026>.

2 Background and Related Work

Our work builds on a body of prior research on web bloat, content similarity, and the environmental costs of the web:

2.1 Over-Fetching and JavaScript Bloat

Several prior works have investigated the gap between delivered web resources and user needs [28, 73, 80]. Existing mechanisms attempt to mitigate this mismatch by lazily loading resources [22, 51] or blocking ads and tracking scripts [72]. However, neither approach provides a general method for selecting the appropriate set of resources for a given task [75]. For instance, archival tasks may have different needs than a person browsing the web [80]. As web crawlers have become more prevalent [63], efforts such as Mark-down for Agents look to optimize for fetching programmatically consumable assets [50].

Prior work has also investigated ‘bloat’ on the web: unused resources, dead code, or tracking scripts that do not affect user experience [6, 29, 46, 62]. In particular, Kupoluyi et al. [46] and others estimate that 70% of JavaScript functions and 55% of shipped code are never invoked [5, 46]. As a result, removing unused JavaScript has become natural for improving performance [15, 46, 49]. Tools like Core Web Vitals [74] and Lighthouse [30] help developers reduce page size and improve performance [14, 66]. However, these tools frequently require developers to expend significant effort to implement [14, 26, 48] or are overly restrictive of their abilities [14].

Previous work offers a narrow view of webpage fidelity, typically adhering to a single dimension [19, 28, 35] or enforcing an overly strict notion of fidelity [80]. However, as we show, similarity along one dimension does not necessarily imply similarity along others. Further, the relevant dimensions vary according to the visitor’s intended use case [50, 80]. We build upon this idea to understand what resources are needed for specific tasks, but we argue that prior work has taken a conservative approach when defining unnecessary content. Typically, web resources are considered unnecessary only when they provide no functionality or user-visible benefit. In contrast, we argue that this definition should be expanded: any web resource that does not contribute to the specific utility function of the web visit should also be considered unnecessary.

2.2 Content Similarity

Determining the importance of a web resource requires measuring how it changes user experience. Prior work has proposed quantitative proxies for measuring webpage fidelity across textual [19, 57, 59], structural [10, 80], visual [35, 79], and functional dimensions [35]. While some studies caution that generalized quantitative metrics do not always match human-perceived differences [35, 80],

researchers have addressed this gap by developing metrics explicitly grounded in human evaluation. In particular, Hashmi et al. [35] introduce Q_{LU}E Structural Similarity (QSS) to measure webpage visual fidelity, and show through a user study that it aligns with human judgments of webpage differences.

2.3 Environmental Cost of the Web

Digital content consumption and website sizes have both grown rapidly in recent years; the median page size increased from 653 KB in 2012 to 2,950 KB in 2026 [38, 64]. Together, these trends have raised environmental [8, 44, 65, 69, 78] and accessibility [33, 34] concerns. In response, recent sustainability efforts have emerged to document and reduce the growing environmental costs of web browsing [36, 54, 65, 76]. The Sustainable Web Design Model (SWDM) estimates the carbon emissions of a page visit as a function of bytes transferred, adoption of green hosting, and other factors [65]. Notably, although SWDM is parameterized by bytes transferred, its estimate includes broader system-level factors, including CPU utilization and embodied emissions [65]. Additionally, it accounts for caching for returning visitors. We use their recommended parameters in our analysis. Our work reveals page size inefficiencies for specific use cases that translate into non-trivial carbon emissions, as quantified by SWDM.

3 Motivating Use Cases

Our work builds on the observation that different use cases on the web have significantly different functionality requirements. We define four *modalities*—distinct modes of access with unique requirements of page utility—that support a variety of use cases. We evaluate our work under four modalities: (1) Text Extraction, (2) Content Archival, (3) Programmatic Automation, and (4) Human Browsing. We select these modalities based on their prevalence in research from the web and Internet measurement communities. While these are not meant to be an exhaustive list, they cover a wide range of workflows.

Text Extraction. Visitors often seek text when visiting a webpage. For example, 9–14% of Android users’ time on the web is spent on News & Media sites [60]. Browsers have introduced tools like Firefox Reader View [56] or Chrome Reading mode [31], which retroactively optimize a page for reading [47]. These tools do not reduce resource usage in their current form, but indicate the popularity and utility of simply extracting relevant text from a page.

Content Archival. Projects like the Internet Archive [43]’s Wayback Machine seek to archive online content. In addition, search engines and AI companies fetch content—including images and video—for indexing and training [2]. In these cases, capturing embedded content is critical, but preserving the interactive components is not. The need to preserve interactivity depends on the archival goal: full replay may require dynamic functionality, but content-focused archival can prioritize the captured text, structure, and media over complex interactions [12].

Programmatic Automation. Measurement studies often employ headless browsers to programmatically load, navigate, and interact with websites [22]. Browsers perform style computation, layout, and rendering internally because many web APIs and JavaScript behaviors depend on the page’s rendered state [41]. However, in

many of these studies, the visual rendering of the website is often disregarded [4, 9, 39]. As such, it is sufficient for headless browsers to preserve all other dimensions of similarity; we use programmatic automation as a motivating example of a workflow that requires the original page’s textual, structural, and interactive content.

Human Browsing. A human visitor expects to receive the full intended experience of a website. However, this does not require the underlying structure to be the same, as long as the experience remains unchanged. For this reason, we also consider the case where the DOM may change due to the inclusion or exclusion of a resource that may have a minimal impact on other aspects of the webpage (e.g., tracking scripts). Critically, users must be able to interact with the site and perform any key actions. For example, when visiting an e-commerce site, users must be able to browse products, add items to a cart, and complete purchases [58]. However, minor structural changes in the DOM hierarchy that affect neither content nor interactivity would not inhibit the user’s experience. Consequently, we model human browsing by selectively loading resources that preserve user-visible behavior (text, visuals, and interactivity) while omitting those that do not contribute to the user’s interaction with the site (structure).

4 Methodology

In this section, we introduce four quantitative measures of *fidelity*: how closely a webpage matches its original form after some of its resources are removed. We use these metrics to evaluate *selectively loaded* websites: page loads for which we only allow loading a subset of web resources. Finally, we describe how we chose which web resources to sequentially include in the ‘selectively loaded’ versions of our webpages and our crawling setup.

4.1 Quantifying Fidelity

We seek to understand resource consumption by selectively loading resources. However, the question of which resources provide utility to a visitor depends on the visitor’s goals. In many cases, visitors do not require the fetched webpage to be identical to the original and require fidelity along a limited set of dimensions.

We identify four dimensions along which fidelity between an original webpage and a selectively loaded version can be evaluated:

- **Textual:** the main visible text content presented on a page
- **Structural:** the structural organization of the page, represented by the Document Object Model (DOM) hierarchy
- **Visual:** the visual appearance of the page, including layout, styling, and media
- **Interactive:** the ability to perform meaningful actions

While these dimensions are not mutually exclusive (e.g., changing the DOM structure of a page is likely to also change its visual appearance), they are designed to capture the aspects of a webpage that a visitor may care about. Furthermore, multiple dimensions of fidelity enable us to take a more nuanced look into webpage utility; for some types of pages, such as news articles, users may care significantly more about conveying the textual content than the visual fidelity. The following paragraphs describe each metric and how we measure it. Each metric is scored on a scale of 0 to 1.

Textual. We measure the similarity of the main readable text. To extract readable text, we use Mozilla Readability [55] (used in Firefox Reader View), which takes HTML as input and outputs the reduced version of the HTML with only the main text content. We then remove HTML tags to keep only the text between them, as this is what is displayed to and read by the visitor. An example of this transformation is given in Figure 1. We then calculate the Jaccard string similarity between the output text of the original website and the selectively loaded website for the textual score. To confirm that our text fidelity metric tracks human judgment, three authors independently rated 60 randomly sampled pairs of selectively loaded webpages. Each pair was presented in a web interface, and raters scored it on a five-point Likert scale ranging from completely different to identical. The authors’ mean ratings correlated strongly with the calculated metric ($r = 0.97$, $p \ll 0.05$).



Figure 1: Example of Extracted Text—We use Readability [55] to infer the main content and remove tags to compare textual content.

Structural. We capture the structural similarity of the DOM trees of the two webpages by calculating the weighted Jaccard similarity between them, where each node represents an HTML element. Our structural metric compares the presence or absence of DOM tree nodes. We assign each node a weight that decays exponentially with its depth in the tree, with shallower nodes contributing more than deeper nodes. Specifically, each parent DOM node has twice the weight of each child. This weighting scheme is intended to reflect the intuition that changes near the root of the DOM typically affect a larger portion of the page than changes to deeply nested leaves. Specific HTML tags are not prioritized over others—nodes are weighted equally if they are siblings, regardless of their element type. Figure 2 illustrates this computation on a webpage that uses an external JS script to insert a list item. To provide grounding for this calculation, we also provide a visual representation of a selectively loaded webpage and its associated structural fidelity score in Appendix B, Figure 12.

Visual. We measure visual similarity using QLU Structural Similarity (QSS) [35] between screenshots of the original and selectively

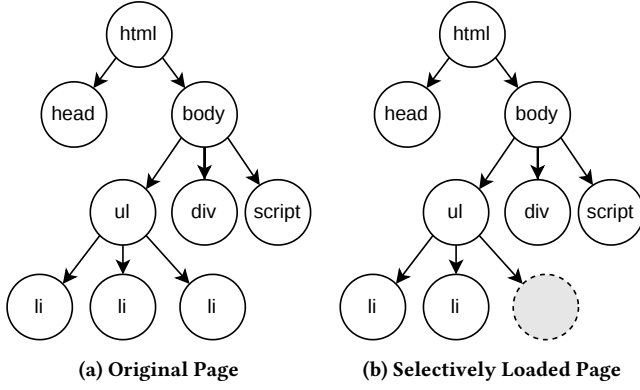


Figure 2: Example of DOM structures of Two Webpages—

In this scenario, the two versions of this webpage would have a structural fidelity score of 0.96. This is calculated by first adding the values of the *union* set of nodes in both trees. The value of each node is $\frac{1}{2^d}$ where d = depth in the tree, with the root having a depth of 0. In our example, this amounts to $S_U = 1 + 2(\frac{1}{2}) + 3(\frac{1}{4}) + 3(\frac{1}{8}) = 3.125$. We then add the values of the *intersection* of nodes in both trees. This amounts to $S_I = 1 + 2(\frac{1}{2}) + 3(\frac{1}{4}) + 2(\frac{1}{8}) = 3$. To get the final score, we calculate $\frac{S_I}{S_U} = 0.96$.

loaded pages. A naive alternative would be to compare the screenshots directly with a pixel-level metric such as SSIM. However, screenshots are sensitive to small layout shifts: minor changes in position can produce large pixel-wise differences even when the pages remain visually similar to a user. QSS reduces this issue by comparing corresponding visual components, such as text regions, images, and buttons, rather than treating the full screenshot as a single image. A component that has merely moved is still matched, so QSS is robust to page-level layout shifts, though comparison within a matched component remains pixel-based. Prior work shows that QSS is stricter than human perception when it comes to visual differences [35], so we use it as a conservative approximation for visual fidelity. We provide an example of the visual fidelity for three screenshots in Appendix B, Figure 12.

Interactive. Unlike the other dimensions, interactivity cannot be inferred directly from page content. Whether a selectively loaded page remains interactive depends on what actions matter for the website. We therefore operationalize interaction fidelity as task completion on site-relevant actions. For example, a video streaming site would place higher value on playing a video than a news website that may prioritize navigating to an article. We describe how we measure interactivity in the next section.

4.2 Measuring Interactivity

To capture this nuance, we take a two-step approach: (1) determining site-relevant tasks and (2) evaluating how the user’s ability to complete these tasks changes as we re-enable resources.

Determining Tasks. To determine tasks, we categorize websites with Cloudflare’s URL Scanner API [18]. We then compile a set of predefined tasks that users commonly perform for the most popular categories. Specifically, we generate tasks for: *News and*

Media, Ecommerce, Shopping & Auctions, and Video Streaming. We specifically consider these categories because they are the most prevalent categories after (a) dropping overly broad categories (e.g., Entertainment and Business), and (b) dropping prohibited/sensitive categories (e.g., Adult Themes and Gambling). The details of this selection strategy are described in Appendix C. Additionally, we combine the Ecommerce and Shopping & Auctions categories into one superclass (Shopping & Ecommerce) due to their large overlap in URLs and functionality. We manually assign the best-fitting category for sites with more than one of these categories.

The generated tasks are derived from the Mind2Web and WebTailBench datasets [7, 23]. We elect to use tasks from these datasets as they are human-annotated and generally accepted benchmarks that provide a baseline for tasks a generalist agent can perform. In our distillation of tasks, we generalize tasks from the datasets to be broadly applicable to the relevant category. For example, instead of ‘purchase Sony headphones from amazon.com’, we generalize this to be ‘purchase a best-selling or featured product’. We also include some manually defined, category-specific tasks to better capture the functionality required of a webpage. This resulted in 15 candidate tasks for Shopping & Ecommerce, and 14 tasks each for News & Media and Video Streaming sites.

We forgo generating a set of ‘general tasks’ since these tasks would not be representative of the core functionality requirements of the website. For instance, testing the navigation to a contact page for an “Entertainment” website would hardly be representative of the user’s intended usage of the website. An alternate idea would be to conduct a user study to determine important tasks, but this presents issues with scalability. We acknowledge that there is a tradeoff between coverage and granularity, and future work can improve upon this. For each task, two authors independently rated it out of five according to how important they deemed them for the given category. The weight assigned to each task is the average of the two authors’ answers. The exhaustive list of categories, tasks, and weights are given in Appendix C, Table 5.

Executing Tasks. After identifying the tasks, we attempt them on the page via an AI browser agent: BrowserUse [11]. We employ BrowserUse due to its strong performance on the WebVoyager benchmark [37] compared to similar browser automation agents. BrowserUse is connected to the same Chrome DevTools Protocol (CDP) session as our Playwright crawler. We load the webpage through Playwright and then allow BrowserUse to attempt the predefined tasks. We then calculate the weighted fraction of tasks that were successfully executed by the agent on the selectively loaded webpage. The task is also verified by an independent judge agent, a practice in line with prior work [7, 23, 40].

We examined a small portion of the agent logs as a spot check and found that the judge could accurately ascertain the success of a task. To avoid penalizing selectively loaded pages for tasks that the agent cannot solve even on the original page, we restrict evaluation to tasks that are completed successfully on the fully loaded baseline page. In the median case, the agent was able to perform nine tasks on a webpage. (Category Breakdown: News & Media (9), Shopping & Ecommerce (9), Video Streaming (9.5)). We opt to use OpenAI’s gpt-oss-120b model for the underlying model powering BrowserUse due to its combination of quick inference speed, low

Modality	Textual	Structural	Visual	Interactive
Text Extraction	✓			
Content Archival	✓	✓	✓	
Programmatic Automation	✓	✓		✓
Human Browsing	✓		✓	✓
Strict Fidelity	✓	✓	✓	✓

Table 1: Fidelity Dimensions By Modality—Modalities generally require a subset of fidelity dimensions.

cost, and task completion accuracy [3].

4.3 Scope of Considered Modalities

We summarize the modalities from Section 3 and the dimensions along which fidelity to the original page is necessary in Table 1. We emphasize that the modalities we consider are not exhaustive, and they can be mixed and matched for other use cases. For example, specific research questions could be answered using only the structural [20] or visual content [32, 45, 47] of the page. In addition, we consider preserving full fidelity along the dimensions of interest in our evaluation. However, it may be sufficient to have partial fidelity in some dimensions for some use cases. For example, a reader tool may want partial visual fidelity in addition to full textual fidelity. Similarly, archival tools may want to preserve some limited interaction.

4.4 Thresholding Fidelity

Webpages naturally have variation across loads, even if the primary content has not changed. For example, video suggestions may be dynamically generated, or advertisements may differ. Thus, it is unfair to expect 100% fidelity across all loads. We set the threshold for an identical load by calculating the fidelity between two separate loads of the original (fully-loaded) webpage. This ensures that natural variations in webpages are taken into consideration. Thus, if a webpage is 90% visually similar to itself upon a full reload, then the selectively loaded webpage needs to be at least 90% visually similar to the original webpage to be considered visually identical.

A selectively loaded page is considered identical along a given dimension—visual, structural, textual, or interactive—if it satisfies the corresponding fidelity threshold. If it satisfies all dimensions, we consider it to have strict fidelity to the original webpage.

4.5 Resource Selection Strategy

In our measurements, we selectively allow certain web resources to load to determine which are essential for rendering the page. A naive approach would be to evaluate every subset of web resources to determine which resources are required to be fetched. However, this quickly becomes intractable: the number of combinations grows exponentially with the number of resources. For instance, the median number of HTTP requests made by sites is 76 [38], yielding 2^{76} combinations. Exhaustively evaluating each of these combinations would be computationally infeasible. Moreover,

repeatedly loading a website at this scale raises ethical concerns, since this could be perceived as a denial-of-service attack.

Instead, we employ a systematic approach that does not try every combination of resources. We define an ordered set of *profiles*, where each profile specifies which external web resources are permitted to be fetched. Profiles that are more likely to preserve page fidelity while reducing resource usage are evaluated first. We evaluate a sequence of increasingly permissive profiles. We begin with a maximally restrictive profile that blocks all external images, stylesheets, fonts, and JavaScript. We then re-enable resource classes in stages: first images, then stylesheets, then fonts. After enabling non-JS resources, we incrementally re-enable external JavaScript: allowing one additional script per load, up to 80 scripts (90% of webpages fetch fewer than 79 external JavaScript resources as of March 2026 [38]).

When a profile satisfies all our fidelity thresholds (i.e., it is strictly identical to the original webpage), the search terminates early, avoiding the evaluation of additional profiles that are unlikely to yield further savings in system resources. In our evaluation, we consider the final visits to be the optimal visits, and report our findings on them. Across all profiles, we apply EasyList [25] rules to block advertising scripts, which are unlikely to contribute to user experience and may vary across reloads. Importantly, our definition of ‘necessity’ is visitor-centric; for the modalities we consider, advertising scripts do not necessarily contribute to the relevant experience. An exception is ad slots, which would be accounted for by our visual similarity metric. We discuss the monetization implications of removing advertising scripts in Section 6. In total, each website is loaded up to 84 times. We discuss the limitations and ethical implications of our resource selection strategy in Section 6 and Appendix A, respectively.

Our inclusion strategy approaches JS conservatively, motivated by prior work showing that JS delivery and execution constitute a substantial share of network and computational usage [21, 27, 68]. Accordingly, we take a finer-grained approach with JS than other web resources; we load JS scripts in their intended fetch order, following the heuristic that site developers would prioritize fetching critical JS first to maximize user QoE. We emphasize that this study is not meant to be an implementation guide on selectively loading webpages, but rather a first-order measurement of the potential benefits achievable under one reasonable strategy. As such, our goal is not to develop a true optimal strategy, and future work may further refine and improve these approaches. Critically, evaluating multiple combinations of web resources may not be the solution, and there may be patterns that emerge to help more intelligently decide whether to fetch a resource in the first place.

4.6 Experimental Setup

We evaluate the potential of system resource savings while preserving fidelity. In this section, we detail our crawling methodology.

To construct our evaluation corpus, we collect websites from multiple popularity tiers. Specifically, we include the CrUX Top 1,000 sites [61], as well as 1,000 randomly sampled websites from three additional rank buckets: 1K–10K, 10K–100K, and 100K–1M (total 4,000 domains). Sampling across these tiers allows us to capture websites with varying levels of popularity.

Popularity rank	All	Interactive	%
1–1K	787	178	22.6
1K–10K	757	155	20.5
10K–100K	693	138	19.9
100K–1M	828	160	19.3
Total	3,065	631	20.6

Table 2: Breakdown of Sites remaining in our corpus after filtering out unsuccessful loads—The ‘Interactive’ column denotes the number of sites for which we did interactivity tests. The final column represents the share of each bucket that underwent interactivity tests.

To aid with our interactivity analysis, we classified the domains in our corpus using Cloudflare’s URL Scanner API [18]. For the modalities in our results that require interactive fidelity, we restrict our analysis to a subset of these domains that had categories for which we were confident in the task selection. This yielded 840 candidate domains. We discuss our candidate domain selection process in Appendix C. We visit each domain’s landing page using an automated headless Chrome browser controlled by Playwright. Additionally, we use Chrome DevTools to inspect and intercept network requests. We store the total bytes on the wire required to load each webpage. The approach we take to determine which resources to intercept is described in Section 4.5.

Chrome DevTools also exposes the CPU time and the IDs of the processes managing the browser instance, which we use to measure the memory usage of the page visits. We measure browser memory usage as the sum of the resident set size (RSS) across Chrome processes associated with the browser instance. The memory and CPU overhead of loading a webpage is then computed as the difference between the memory usage and CPU time of a fresh tab and that of the tab after the page has fully loaded. We additionally parallelize across ten servers with 16GB RAM and 16 cores each. There was no resource contention during these visits, and the memory, CPU, and network usage remained well below the capacity of our infrastructure.

For each page visit, we wait for 20 seconds for the ‘load’ event [52] to fire. We wait an additional 1.5 seconds after this event to allow rendering to complete. If the page does not successfully load within 20 seconds (i.e., it does not return a 200 HTTP status code), we remove it from our corpus. As a result, 3,065 domains remained in our corpus (of which 631 underwent interactivity tests). Table 2 breaks down the number of sites that remained in our corpus by popularity rank.

5 Results

We now evaluate what selective loading for specific modalities reveals about unnecessary resources. We organize our findings around five questions:

- (1) How much web bloat is revealed when considering modality?
- (2) Which external resources are required to preserve fidelity?
- (3) Which fidelity dimensions constrain the reducibility of sites?
- (4) Where is reducibility concentrated across popularity and type?
- (5) Can fidelity along one dimension predict fidelity along others?

5.1 Modality Reveals Additional Bloat

We measure how much of a page’s resource usage can be eliminated under strict fidelity and under each of the modality-specific definitions from Section 3, then consider what the resulting savings imply for the sustainability and affordability of web access.

Strict Fidelity. We begin by evaluating against a baseline for removing unnecessary resources—strict fidelity—which requires the selectively loaded page to preserve textual, structural, visual, and interactive fidelity. Remarkably, 42% of websites can maintain strict fidelity to their fully fetched counterparts while consuming fewer system resources. For these websites, we observe a 14% median reduction in page size, along with median savings of 45% in CPU time and 8% in memory usage (Appendix D, Figure 13).

Modality-specific Fidelity. However, once fidelity is defined according to the visitor’s modality, substantially more resources can be removed. For instance, we can preserve full textual fidelity for 93% of websites while selectively loading a subset of resources. Furthermore, the median reduction in network usage is 98% (Figure 3a). To put this in perspective, among the pages identified as textually identical, the median page size drops from 2,142 KB in the original fetch to just 43 KB when selectively loaded. Note that this is not merely the size of the textual content, but the *total page size required to produce textual content*, which includes all JavaScript needed to dynamically generate it. In addition, this approach saves 76% of CPU time and 53% of memory.

This remains true—but to a lesser degree—for other modalities. For content archival, which additionally requires structural and visual fidelity, 56% of websites remain identical. For programmatic automation and human browsing, which preserve interactivity, 57% and 54% of websites, respectively, can be reduced without losing the relevant fidelity dimensions.

Eligibility vs magnitude. Our results reveal two distinct aspects of reducibility: 1) whether a page can reduce resources without any change in relevant fidelity, and 2) how much can be reduced once it does. Text extraction has both high eligibility and a high magnitude of reduction: most pages can preserve textual content, and the amount removed is substantial. On the other hand, content archival is able to preserve fidelity for over half of the websites but has a more moderate degree of byte savings. Human browsing and programmatic automation lie somewhere in the middle—they preserve more aspects of fidelity than text extraction, but still expose unneeded resources that strict fidelity misses (Figure 4).

While we see differences in page size reduction, CPU reductions remain large across all modalities, ranging from 39% to 76% in the median case (Figure 3), suggesting that selectively removed resources are often disproportionately expensive to process even when page size reductions are more modest.

Sustainability. The reduction in system resources has implications for the sustainability of web visits. Using SWDM (Section 2.3), we approximate the benefits of selective loading. With the browsing patterns we observe today, human browsing accounts for 68.8% of traffic, and content archival accounts for an additional 6.9% [67]. We find there is potential to reduce human browsing emissions by 18% and content archival emissions by 14% (Figure 4). We use these traffic shares to scale our measured byte reductions to an annual estimate of total data transferred [42]. For each modality,

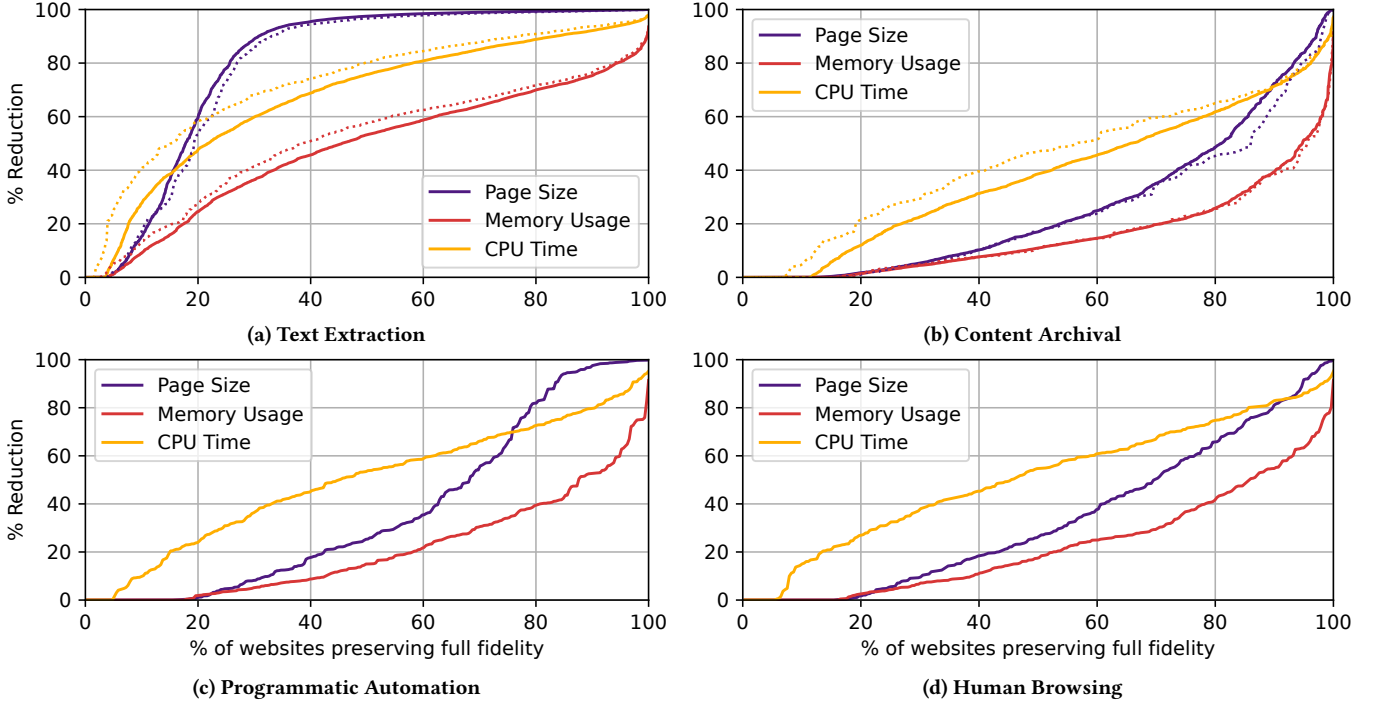


Figure 3: Resource Reductions by Fidelity Requirement—We plot the resource reduction of selectively loaded pages that could be reduced and preserve the relevant fidelity dimensions. Because interactivity was tested only for programmatic automation and human browsing, those panels use that tested subset; dotted lines in text extraction and content archival show the same subset for comparison.

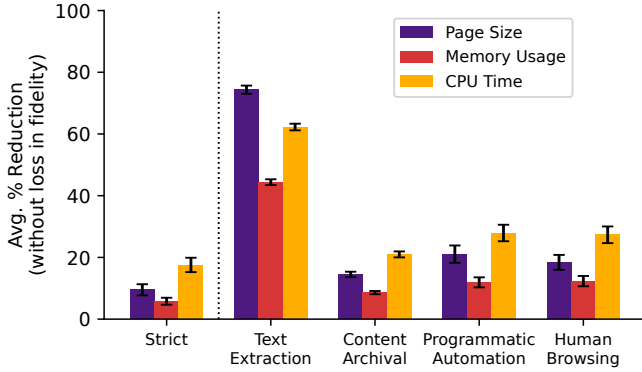


Figure 4: Average Resource Reductions by Modality—Error bars represent 95% confidence intervals. Reductions are averaged over all considered websites. When no selectively loaded version satisfies the modality-specific fidelity criteria, the original fully loaded page is used, yielding a 0% reduction. The magnitude of reduction depends on the fidelity dimensions required by the visit. Text extraction exposes the largest possible reductions.

we multiply total annual Internet traffic by the modality’s traffic share and by the corresponding page size reduction, then convert the avoided bytes to emissions using SWDM (Section 2.3).

Under this model of potential savings, we estimate a possible emissions reduction of 159 million metric tons of CO_2e per year

without loss of utility. To contextualize this number, the average passenger vehicle in the United States emits roughly 400 gCO_2 per mile driven [71]. This estimate should be interpreted as an upper-bound opportunity rather than a direct prediction, since it assumes that our measured reductions generalize across traffic in each modality. Ultimately, these estimated emissions stem from wasted system utilization. Pages can maintain the required fidelity for several modalities without fetching many of their resources. On the other hand, if we consider only the reduction based on strict fidelity, the unnecessary emissions amount to 86.5M metric tons/year. In other words, a strict fidelity definition misses nearly half of the avoidable emissions.

Affordability. The potential reduction in page size also has implications for the affordability of web access. For regions where limited data plans are the norm, every byte downloaded to access a website has an associated monetary cost. The average reduction in page size required to keep all countries within the affordability budget set by the UN Broadband Commission for a standard data-only plan is 79% [34]. Our results show that we can modestly approach this goal without impacting the user-relevant content: for human browsing, the average reduction is 18% of the original webpage (Figure 4). This is despite preserving the full fidelity dimensions relevant to a human browsing experience; we envision a use case where limited fidelity is a preferable outcome for affordability.

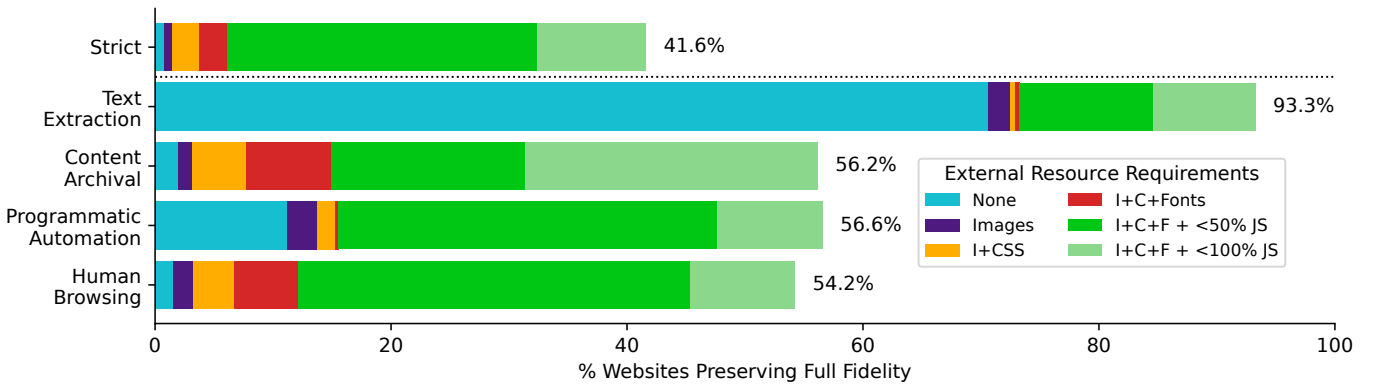


Figure 5: Share of websites that preserve full fidelity for a given modality without external resources—I=Images, C=CSS, F=Fonts, JS=JavaScript. Hatched bars represent profiles without JS. Even in cases where external JS is required, websites often only need less than half of the scripts.

5.2 Resource Requirements

We next examine the external resources needed to preserve fidelity. Our loading strategy progressively re-enables resource classes, so the first successful stage for a website reveals the minimum resource requirements for that modality among the profiles we consider. External resources are typically unnecessary for text extraction: in 76% of cases, we do not need any of the four external resources (images, CSS, fonts, and JS) we consider. This explains the massive reduction in page size for this modality.

Interestingly, a small fraction of websites (2%) required images to preserve the extracted textual content. We manually inspected these pages and found that externally loaded images occupied a substantial portion of the viewport. We hypothesize that loading these images changed the layout or element visibility in the DOM, which in turn changed the text extracted by Mozilla Readability. Even when JS is needed, often only a small fraction is required for full fidelity. Among the 21% of websites that require some external JS to remain textually identical, the median webpage only needs 41% of its scripts (Figure 5).

Despite being constrained by structural and visual fidelity, more than a quarter (26%) of the websites that can be reduced require no external JS for content archival. Among websites that do require JS, the median site needs 65% of its original JS requests. A small fraction of websites (3%) can preserve archival fidelity without any of the external resources we consider. This is disproportionately uncommon for Shopping & Ecommerce websites: although they make up 10% of the dataset, they account for only 4% of websites that preserve full archival fidelity without external resources.

For programmatic automation, which preserves textual, structural, and interactive fidelity but does not require visual fidelity, 20% of selectively loaded websites benefit from blocking all considered external resources. Even when external JS is needed, only a small fraction is required: the median page needs 22% of its original JS requests. This suggests that even when JavaScript is essential for the required interactions, most of a page’s scripts are not. Many pages preserve the required automation behavior with only a subset of scripts, likely because some external scripts support presentation,

analytics, ads, or auxiliary functionality rather than the relevant tasks.

Many scripts fetched during normal human browsing are not necessary to preserve the user-visible and interactive experience. Notably, in this modality, JavaScript is often over-fetched: among the 77% of websites that require some external JS, the median page needs only 20% of the original scripts.

5.3 Why Reductions Fail

The previous subsection shows that possible reductions are largest for modalities where visual fidelity is not required. We now examine why this is the case by looking at how fidelity changes as resource types are progressively re-enabled.

Visual fidelity is the hardest dimension to recover. When external resources are blocked, visual fidelity is substantially lower than the other metrics, leaving more fidelity to recover as resources are re-enabled (Figure 6). This helps explain why programmatic automation generally achieves larger page size reductions than content archival (Figure 4). This suggests that visual fidelity can be a stronger constraint on system resources than interactivity.

Re-enabling images improves visual fidelity less than re-enabling CSS and fonts, suggesting that layout and styling matter more to perceived fidelity than image content. This gap may be partly an artifact of how QSS scores matched components: because comparison within a component is pixel-based, a substituted font shifts every glyph inside each text region and incurs a large penalty even when the rendering remains legible and close to the original. On the other hand, textual and structural fidelity remain relatively high and stable without any external JS. Interactive fidelity also remains high even when external JS is blocked, indicating that critical functionality is often preserved through inline scripts and browser-native behavior.

These results show where selective loading fails. Visual fidelity is especially constraining because it depends on multiple resource classes simultaneously: layout, styling, fonts, images, and sometimes JS-driven rendering. This also means that resource necessity is not binary. The same web resource can be irrelevant for one modality and essential for another.

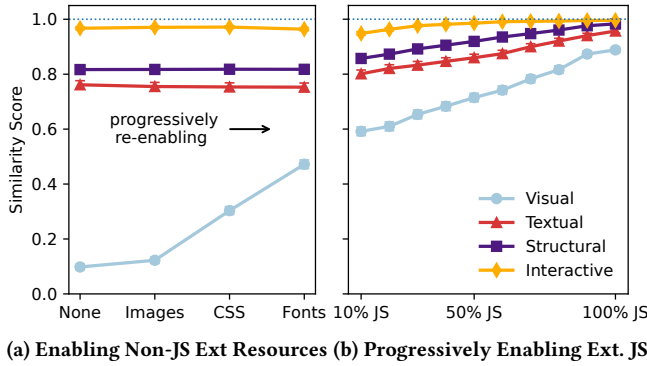


Figure 6: Fidelity as External Resources Are Re-enabled—Error bars represent 95% confidence intervals. Textual, structural, and interactive fidelity remain relatively stable until external JavaScript is introduced, while visual fidelity is harder to recover.

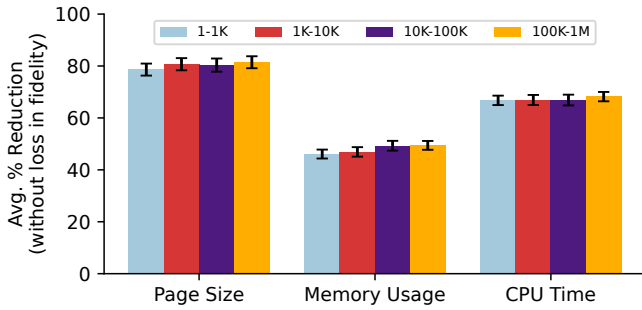


Figure 7: Textual trade-offs by Site Popularity—Reductions remain high across popularity ranks. Error bars represent 95% confidence intervals.

5.4 Where Bloat Is Concentrated

We next examine whether reducibility is concentrated in particular parts of the web. We look at the popularity of the websites as defined by the CrUX buckets and the categories defined in Section 4.1. Across modalities, trends depend on which fidelity dimensions must be preserved. For text extraction, reductions are broadly available. The popularity of a website does not determine how much reduction we see while preserving full textual fidelity (Figure 7). All categories we consider are highly reducible, with News & Media websites showing somewhat higher reductions (Appendix D, Figure 14a). This reinforces the potential of reader-mode tools to reduce resource consumption while preserving relevant textual content.

The trend is different for content archival, which additionally preserves visual fidelity, where possible reduction increases with website popularity (Figure 8). This suggests that popular sites include substantial interactive components that can be dropped when the goal is to preserve viewable content rather than interaction. For programmatic automation, which preserves textual, structural, and interactive fidelity while relaxing visual fidelity, we do not observe a significant popularity trend (Appendix D, Figure 15a). However, Video Streaming sites show the highest potential for overall reduction (Figure 9). This follows the intuition that streaming sites include substantial resources tied to visual presentation, such as

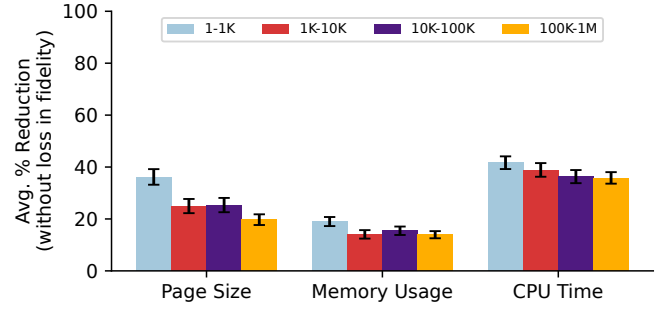


Figure 8: Content Archival trade-offs by Site Popularity—More popular websites have greater potential for system resource reduction. Error bars represent 95% confidence intervals.

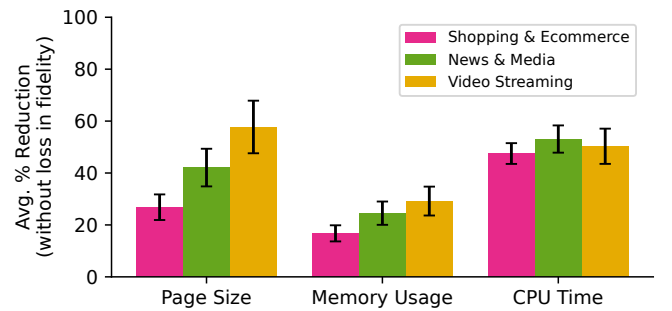


Figure 9: Programmatic Automation Trade-offs by Category—Video Streaming websites are highly reducible when visual fidelity is not required. Error bars represent 95% confidence intervals.

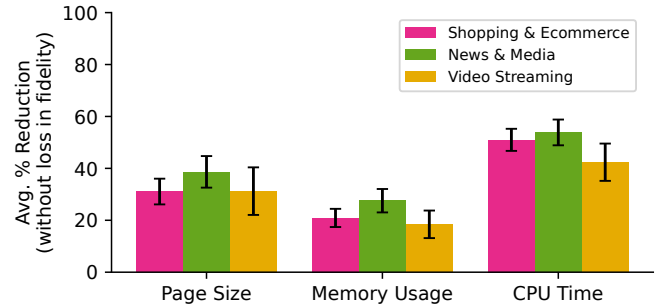


Figure 10: Human Browsing Trade-offs by Category—News & Media has the largest reductions while preserving visual and interactive fidelity. Error bars represent 95% confidence intervals.

thumbnails and rich media assets, that are unnecessary when the browser is used for non-visual automated tasks.

For human browsing, where textual, visual, and interactive fidelity must be preserved, we see that News & Media websites show the largest average reductions (Figure 10). Video Streaming websites remain frequently eligible for some reduction, but yield smaller savings since visual fidelity is required. This distinction shows that eligibility and magnitude are different: websites may often admit some reduction without allowing large reductions.

Overall, these results show that web bloat is not concentrated in a single type of site (summarized in Table 3). Popularity matters

Modality	Visual?	Interactive?	Popularity Trend	Category Trend
Text Extraction	✗	✗	No significant difference	All categories highly reducible
Content Archival	✓	✗	More reducible when popular	All categories moderately reducible
Programmatic Automation	✗	✓	No significant difference	Video Streaming most reducible
Human Browsing	✓	✓	No significant difference	News & Media most reducible

Table 3: Resource-reduction Trends Across Modalities—The concentration of reducible resources depends on which fidelity dimensions must be preserved.

only for content archival. Category matters for some modalities, but the relevant category changes depending on whether visual or interactive fidelity is relevant.

5.5 Relationships Between Measures of Fidelity

The previous sections show how different modalities require different fidelity dimensions, and how that affects how we consider bloat on the web. We now ask whether one dimension is enough to infer the others. If fidelity dimensions were closely related, future systems could optimize for one metric and assume the others were preserved. Our results show that this is not the case, and that modality-aware reasoning must explicitly account for these dimensions to better combat web bloat.

No subset of fidelity dimensions accurately predicts an omitted dimension. To show this, we first calculate the pairwise Pearson correlations between fidelity metrics across all the stages we evaluate (Figure 11a). There is very little correlation, if any, between the metrics we consider. This shows that determining fidelity using only one dimension is inadequate. The highest correlation we see is between textual and structural fidelity ($r = 0.32$, $p \ll 0.05$). This matches our intuition that the inclusion or exclusion of a DOM subtree often corresponds to what text is visible to the user (both of which are directly determined by the HTML). Regardless, this is a relatively low correlation.

A limitation of Pearson correlation is that it only captures linear pairwise relationships. To address this, we additionally measure how well each fidelity metric can be predicted from the remaining metrics using both linear regression and nonlinear random forest models. We evaluate predictability using cross-validated R^2 , which measures the fraction of variance that can be explained by the remaining metrics (e.g., $R^2 = 0.5$ would imply half of the variation in one metric can be predicted by the other metrics). While linear modeling showed little to no predictability between metrics, the nonlinear model showed moderate shared structure. However, even under nonlinear modeling, no single dimension can be fully predicted by the others — the highest observed R^2 is 0.57, which means that 43% of the variance remained unpredictable (Figure 11b). These results suggest that the dimensions are partially related but not redundant.

The broader implication of this correlation analysis is that selective fetching strategies must explicitly account for dimensions that are relevant to them, as optimizing for one will not necessarily optimize for others.

5.6 Summary

Overall, we find that selective loading according to modality exposes large reductions in loaded page size, CPU time, and memory usage. Modality-aware measurement reveals substantially more unnecessary resource usage than our baseline, indicating that bloat depends on which fidelity dimensions a visit requires. Further, many webpages can preserve the relevant experience with few external resources, and even JS-dependent pages often need only a subset of their scripts. Visual fidelity is the strongest constraint on reduction, since layout, styling, fonts, images, and JavaScript-driven rendering can jointly affect whether a page remains visually identical. Notably, reducibility is not concentrated in one class of websites: popularity and category matter differently depending on the modality. Finally, because fidelity dimensions are related but not redundant, future selective loading systems must reason explicitly about which dimensions matter for a given visit rather than relying on a single proxy for page fidelity.

6 Discussion

In this section, we discuss the high-level takeaways and limitations of our work.

Recommendations and Future Work. Our results suggest that web visits would benefit from defining unnecessary resource usage by the visitor’s access modality. By capturing this complexity, we show that the problem of web bloat is more pronounced than prior work suggests. More broadly, our findings point to the limits of the web’s current one-size-fits-all delivery model, in which similar bundles of resources are served to visitors with very different goals. Future web systems may be better served by a more differentiated model of delivery, where the resources fetched are better aligned with the fidelity requirements of the visitor or website category. Client-side request interception is already achievable with existing browser automation tools, but the hard problem is identifying *which* few resources matter before fetching, which motivates a cooperatively maintained resource necessity registry.

On the server side, we recommend a mechanism for a client to communicate which modality it is using. Markdown for Agents [50] is one such example that is deployed, but we advocate for a more general scheme covering other modalities. Since fidelity dimensions are related but not redundant, this differentiated model must account for the specific dimensions relevant to a given modality rather than optimizing for a single proxy for fidelity. Such an approach could reduce the cost of archival, improve the efficiency of crawlers and measurement pipelines, and make web access more feasible in

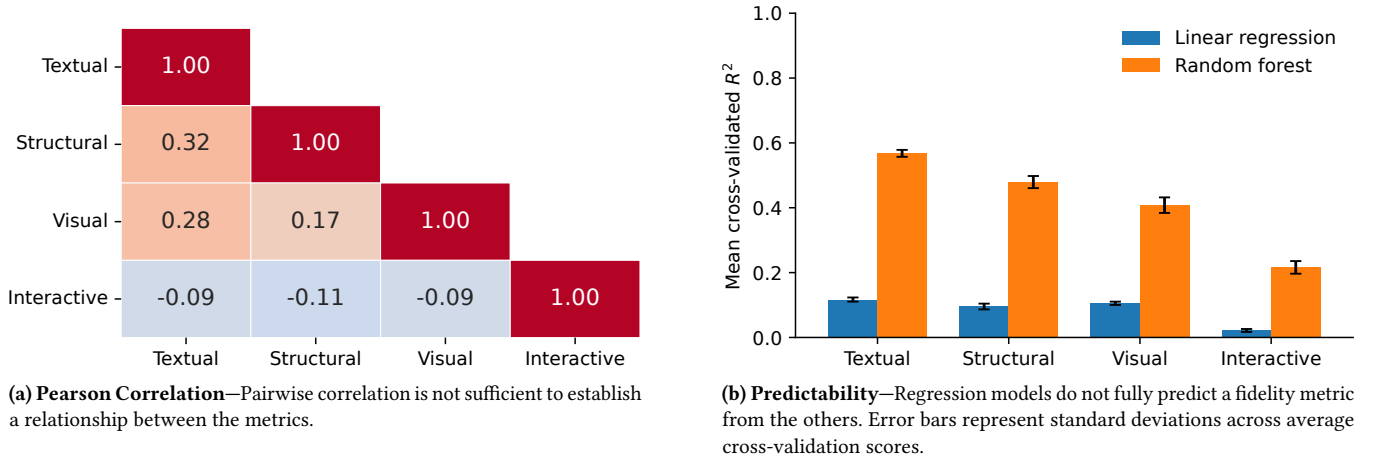


Figure 11: Relationships between measures of fidelity — Fidelity along one dimension does not necessarily imply fidelity along others.

resource-constrained environments. These considerations are especially pressing as automated clients, including AI data collection pipelines and agentic AI browsers, account for an increasing share of web traffic and its associated environmental cost.

Automated Traffic. The implications of web bloat are especially important for non-visual automation. Our results show that this modality can achieve substantial reductions in system resources. Automated traffic occurs at a massive scale: roughly one-third of HTTP requests originate from bots [17] and Cloudflare reported blocking 416 billion AI bot requests in five months across its network alone [77]. Despite blocking malicious bot traffic, 8% of the bot traffic Cloudflare sees is from verified bots [16]. Even at this scale, small per-visit inefficiencies from verified bots may accumulate quickly, potentially surpassing the emissions of millions of miles driven per month. This suggests that selective fetching could be particularly useful for crawlers and measurement pipelines, which often do not require the full visual webpage.

Reasons for Over-Fetching. Websites commonly fetch resources that are unnecessary for a visitor’s specific modality because the web’s delivery model is largely one-size-fits-all. In practice, website operators prefer over-delivery to under-delivery: fetching more resources reduces the risk of breakage, preserves compatibility across visitors, and ensures that monetization mechanisms such as ads, analytics, and tracking are still served. As a result, resources that are unnecessary for the visitor’s task may still be valuable to the publisher’s business model.

This mismatch is especially visible for automated clients, whose needs often differ substantially from those of human visitors, yet who are commonly served the same resources. In such cases, automated clients are not the intended audience for advertisements or other engagement-oriented content, nor do they constitute legitimate traffic [53], yet they still incur the cost of fetching resources that support them. In this sense, over-fetching is not only a technical inefficiency, but also a consequence of the incentives and delivery assumptions of the web. The trade-off between efficiency and publisher constraints on monetization should be considered for realistic deployments of selective loading.

System-Constrained Loading. Our evaluation of selectively loaded websites fixes the fidelity dimensions relevant to each modality and loads web resources until the corresponding fidelity thresholds are satisfied. Notably, this paradigm can be restructured: fidelity can be scored under a fixed budget of system resources. This is especially relevant in resource-constrained environments where users may have limited data plans, marginal network bandwidth, or memory-constrained devices. In such settings, selective loading could expose an explicit trade-off between resource consumption and modality-specific utility.

Other Benefits. In this work, we provide a measurement of unnecessary resources, in terms of network bytes, CPU time, memory usage, and carbon emissions. Web bloat has other implications. For example, unnecessary web resources increase the attack surface and privacy exposure associated with executing complex scripts. Similarly, reducing unnecessary resources improves auditability: pages with fewer dependencies are easier to inspect, debug, and reason about. Moreover, reducing webpage size often improves performance, since fewer bytes need to be transferred and rendered. Lastly, this can reduce strain on content providers and improve reliability. Content providers would need to handle fewer requests if pages were selectively loaded.

Limitations. There are several limitations in our study. First, interactivity is highly subjective, and usage patterns on specific websites can vary drastically. We approximate this by specifying categories and considering a broad range of tasks. Future work can improve this by conducting user studies to find which functions are necessary for a given website. Notably, we did not attempt to log in on sites, which may have changed the outcome of some tasks. In addition, interactivity is broad and could be split into multiple sub-dimensions. Second, we use a simple, heuristic-based strategy for deciding which resources to load. In particular, we consider a fixed set of resource categories and evaluate them in a fixed order. While this is sufficient for measuring the potential benefits of selective loading, it is not necessarily the optimal strategy for doing so. Future work can improve upon this by using finer-grained dependency analysis or more adaptive policies for deciding

resource necessity for a given modality. Third, we conduct our measurements with caching disabled. This is a safe assumption for some modalities; automated agents, crawlers, and archivers typically visit each origin from rotating infrastructure, resulting in little cache reuse in practice. For repeat human visitors, our page size reductions should be interpreted as an upper bound. Notably, however, our sustainability estimate (Section 5.1) already accounts for caching.

7 Conclusion

We provide the first modality-aware measurement of unnecessary resource usage on the web. To do so, we define four modalities characterized by combinations of four quantitative measures of fidelity. We find that a baseline that does not reason about access modality underestimates the scale of unnecessary resource usage, obscuring potential opportunities to reduce network transfer, system utilization, and associated carbon emissions. Additionally, we uncover how reducibility varies by modality, popularity, and website category, suggesting that unnecessary resource usage is shaped by both the characteristics of the page and the means by which it is accessed. Finally, we show that subsets of fidelity dimensions cannot predict the remaining ones, and discuss the potential of future systems to reduce the gap between resources that are truly required and resources that are fetched.

Acknowledgments

We thank the shepherd, reviewers, and members of the Empirical Security Research Group (Kimberly Ruth, Agur Adams, Michael Paper, Thea Rossman, and Phillip Stephens) for their insightful comments. We thank Sarah Lowry for her feedback on the sustainability estimates. This work was supported in part by the National Science Foundation under Grant Numbers #2319080 and #2540803, as well as a Sloan Research Fellowship and a Stanford Graduate Fellowship. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the funding organizations.

References

- [1] Muhammad Abdullah, Zafar Ayyub Qazi, and Ihsan Ayyub Qazi. 2022. Causal impact of Android go on mobile web performance. In *ACM Internet Measurement Conference*.
- [2] Mohammad Mahdi Abootorabi, Amirhosein Zobeiri, Mahdi Dehghani, Mohammadali Mohammadkhani, Bardia Mohammadi, Omid Ghahroodi, Mahdiah Soylemani Baghshah, and Ehsaneddin Asgari. 2025. Ask in Any Modality: A Comprehensive Survey on Multimodal Retrieval-Augmented Generation. arXiv:2502.08826 [cs.CL] <https://arxiv.org/abs/2502.08826>
- [3] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, et al. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925 <https://arxiv.org/abs/2508.10925>
- [4] Suood Alroomi and Frank Li. 2023. Measuring Website Password Creation Policies At Scale. In *ACM SIGSAC Conference on Computer and Communications Security*.
- [5] Abdul Haddi Amjad, Nishu Goel, Barry Pollard, Onur Güler, and Nurullah Demir. 2024. *JavaScript*. HTTP Archive, Chapter 1. doi:10.5281/zenodo.14962516
- [6] Abdul Haddi Amjad, Zubair Shafiq, and Muhammad Ali Gulzar. 2023. Blocking JavaScript without Breaking the Web: An Empirical Investigation. *Privacy Enhancing Technologies Symposium* (2023).
- [7] Ahmed Awadallah, Yash Lara, Raghav Magazine, Hussein Mozannar, Akshay Nambi, Yash Pandya, Aravind Rajeswaran, Corby Rosset, Alexey Taymanov, Vibhav Vineet, Spencer Whitehead, and Andrew Zhao. 2025. Fara-7B: An Efficient Agentic Model for Computer Use. <https://arxiv.org/abs/2511.19663>
- [8] Janna Axenbeck, Stefanie Kunkel, Joris Blain, and Francis Charpentier. 2026. Between 2010 and 2021, global emissions from digital technologies were largely obscured in greenhouse gas emission accounting standards. *Communications Sustainability* (2026). doi:10.1038/s44458-025-00022-6
- [9] Paschalis Bekos, Panagiotis Papadopoulos, Nicolas Kourtellis, and Michalis Pylchrounakis. 2025. PIIxel Leaks: Passive Identification of Personally Identifiable Information Leakage through Meta Pixel. In *ACM SIGSAC Conference on Computer and Communications Security*.
- [10] Sacha Brisset, Romain Rouvoy, Lionel Seinturier, and Renaud Pawlak. 2023. SFTM: Fast matching of web pages using Similarity-based Flexible Tree Matching. *Information Systems* (2023). doi:10.1016/j.is.2022.102126
- [11] browser-use contributors. 2026. browser-use: Make Websites Accessible for AI Agents. <https://github.com/browser-use/browser-use>. GitHub repository, accessed 2 March 2026; MIT License.
- [12] Justin F. Brunelle, Michele C. Weigle, and Michael L. Nelson. [n. d.]. Archival Crawlers and JavaScript: Discover More Stuff but Crawl More Slowly. In *ACM/IEEE Joint Conference on Digital Libraries*.
- [13] Michael Butkiewicz, Harsha V. Madhyastha, and Vyas Sekar. 2011. Understanding website complexity: measurements, metrics, and implications. In *ACM Internet Measurement Conference*. <https://doi.org/10.1145/2068816.2068846>
- [14] Center of Expertise for Digital Platform Regulation (PEReN). 2022. Google's AMP: From Addiction to Withdrawal? Technical Report. https://www.peren.gouv.fr/rapports/2023-01-03%20-%20Eclairage-sur-AMP_EN.pdf Accessed: 2026-04-25.
- [15] Moumena Chaqfeh, Russell Coke, Jacinta Hu, Waleed Hashmi, Lakshmi Subramanian, Talal Rahwan, and Yasir Zaki. 2022. JSAnalyzer: A Web Developer Tool for Simplifying Mobile Web Pages through Non-critical JavaScript Elimination. *ACM Trans. Web* (2022).
- [16] Cloudflare. 2024. *Application Security report: 2024 update*. <https://blog.cloudflare.com/application-security-report-2024-update/> Accessed: 2026-04-25.
- [17] Inc. Cloudflare. 2026. Bot vs. Human worldwide. <https://radar.cloudflare.com/traffic#bot-vs-human> Cloudflare Radar.
- [18] Cloudflare, Inc. 2025. Domain Categories. <https://developers.cloudflare.com/cloudflare-one/traffic-policies/domain-categories/>. Cloudflare One documentation. Accessed: 2026-03-26.
- [19] Riccardo Coppola, Robert Feldt, Michel Nass, and Emil Alégroth. 2025. Ranking approaches for similarity-based web element location. *Journal of Systems and Software* 222 (2025), 112286. doi:10.1016/j.jss.2024.112286
- [20] Igino Corona, Battista Biggio, Matteo Contini, Luca Piras, Roberto Corda, Mauro Mereu, Guido Mureddu, Davide Ariu, and Fabio Roli. 2017. DeltaPhish: Detecting Phishing Webpages in Compromised Websites. In *Computer Security – ESORICS 2017*, Simon N. Foley, Dieter Gollmann, and Einar Snekkenes (Eds.). Springer International Publishing, Cham, 370–388.
- [21] João De Macedo, Rui Abreu, Rui Pereira, and João Saraiva. 2022. WebAssembly versus JavaScript: Energy and Runtime Performance. In *International Conference on ICT for Sustainability*.
- [22] Nurullah Demir, Matteo Große-Kampmann, Tobias Urban, Christian Wressnegger, Thorsten Holz, and Norbert Pohlmann. 2022. Reproducibility and Replicability of Web Measurement Studies. In *ACM Web Conference*.
- [23] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. arXiv:2306.06070 [cs.CL] <https://arxiv.org/abs/2306.06070>
- [24] Zakir Durumeric, David Adrian, Phillip Stephens, Eric Wustrow, and J Alex Halderman. 2024. Ten years of zmap. In *ACM Internet Measurement Conference*.
- [25] EasyList Authors. [n. d.]. EasyList. <https://easylist.to/>. Accessed: 2026-03-17.
- [26] Todd H. Gardner. 2025. The Limitations of Lighthouse. <https://requestmetrics.com/web-performance/the-limitations-of-lighthouse/>. Accessed: 2026-04-25.
- [27] Ayush Goel, Jingyuan Zhu, Ravi Netravali, and Harsha V. Madhyastha. 2022. Java: Web Archival in the Era of JavaScript. In *USENIX Symposium on Operating Systems Design and Implementation*.
- [28] Ayush Goel, Jingyuan Zhu, Ravi Netravali, and Harsha V. Madhyastha. 2024. Sprinter: speeding up high-fidelity crawling of the modern web. In *USENIX Symposium on Networked Systems Design and Implementation*.
- [29] Hossein Golestani, Scott Mahlke, and Satish Narayanasamy. 2019. Characterization of Unnecessary Computations in Web Applications. In *IEEE International Symposium on Performance Analysis of Systems and Software*.
- [30] Google Chrome Developers. 2025. Introduction to Lighthouse. <https://developer.chrome.com/docs/lighthouse/overview>.
- [31] Google Chrome Help. [n. d.]. Use Reading mode in Chrome. <https://support.google.com/chrome/answer/14218344?hl=en>. Accessed April 20, 2026.
- [32] Ralf Gundelach and Dominik Herrmann. 2023. Cookiescanner: An Automated Tool for Detecting and Evaluating GDPR Consent Notices on Websites. In *International Conference on Availability, Reliability and Security*.
- [33] Rumaisa Habib, Aimen Inam, Ayesha Ali, Ihsan Ayyub Qazi, and Zafar Ayyub Qazi. 2023. A First Look at Public Service Websites from the Affordability Lens. In *ACM Web Conference*.
- [34] Rumaisa Habib, Sarah Tanveer, Aimen Inam, Haseeb Ahmed, Ayesha Ali, Zartash Afzal Uzmi, Zafar Ayyub Qazi, and Ihsan Ayyub Qazi. 2023. A Framework for Improving Web Affordability and Inclusiveness. In *ACM SIGCOMM Conference*.
- [35] Waleed Hashmi, Moumena Chaqfeh, Lakshminarayanan Subramanian, and Yasir Zaki. 2022. QJUE: A Computer Vision Tool for Uniform Qualitative Evaluation of Web Pages. In *ACM Web Conference*.

- [36] Vassiliki Hatzis, B. Barla Cambazoglu, and Iordanis Koutsopoulos. 2016. Optimal Web Page Download Scheduling Policies for Green Web Crawling. *IEEE Journal on Selected Areas in Communications* 34, 5 (2016), 1378–1388. doi:10.1109/JSAC.2016.2520246
- [37] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. arXiv:2401.13919 [cs.CL] <https://arxiv.org/abs/2401.13919>
- [38] HTTP Archive. 2026. State of the Web. <https://httparchive.org/reports/state-of-the-web>
- [39] Maxime Huyghe, Walter Rudametkin, and Clément Quinton. 2025. FP-Rainbow: Fingerprint-Based Browser Configuration Identification. In *ACM Web Conference*.
- [40] Jayoo Hwang, Xiaowen Zhang, and Vedant Padwal. 2026. WebChallenger: A Reliable and Efficient Generalist Web Agent. arXiv:2606.10423 [cs.CL] <https://arxiv.org/abs/2606.10423>
- [41] Landon Iannamico. [n. d.]. What Is a Headless Browser? <https://www.nimbleway.com/blog/what-is-a-headless-browser>. Published by Nimbleway, accessed 2026-04-09.
- [42] International Telecommunication Union. 2025. Internet traffic. *Facts and Figures 2025*. <https://www.itu.int/itu-d/reports/statistics/2025/10/15/ff25-internet-traffic/>. Accessed: 2026-04-29.
- [43] Internet Archive. [n. d.]. Internet Archive. <https://archive.org/>. Accessed: 2026-04-03.
- [44] Robert Istrate, Victor Tulus, Robert N. Grass, Laurent Vanbever, Wendelin J. Stark, and Gonzalo Guillén-Gosálbez. 2024. The environmental sustainability of digital content consumption. *Nature Communications* 15 (2024), 3724. doi:10.1038/s41467-024-47621-w
- [45] Fujiao Ji, Kiho Lee, Hyungjoon Koo, Wenhao You, Euijin Choo, Hyoungshick Kim, and Doowon Kim. 2025. Evaluating the effectiveness and robustness of visual similarity-based phishing detection models. In *Proceedings of the 34th USENIX Conference on Security Symposium (Seattle, WA, USA) (SEC '25)*. USENIX Association, USA, Article 165, 20 pages.
- [46] Jesutofunmi Kupoluyi, Moumena Chaqfeh, Matteo Varvello, Russell Coke, Waleed Hashmi, Lakshmi Subramanian, and Yasir Zaki. 2022. Muzeel: assessing the impact of JavaScript dead code elimination on mobile web performance. In *ACM Internet Measurement Conference*.
- [47] Qisheng Li, Meredith Ringel Morris, Adam Fourney, Kevin Larson, and Katharina Reinecke. 2019. The Impact of Web Browser Reader Views on Reading Speed and User Experience. In *CHI Conference on Human Factors in Computing Systems*.
- [48] Paweł Madeja. 2026. The Rise and Fall of AMP: Why You Should Remove It and Never Look Back. <https://coditive.com/blog/the-rise-and-fall-of-amp/>. Accessed: 2026-04-25.
- [49] Ivano Malavolta, Kishan Nirghin, Gian Luca Scoccia, Simone Romano, Salvatore Lombardi, Giuseppe Scanniello, and Patricia Lago. 2023. JavaScript Dead Code Identification, Elimination, and Empirical Assessment. *IEEE Transactions on Software Engineering* 49, 7 (2023), 3692–3714. doi:10.1109/TSE.2023.3267848
- [50] Celso Martinho and Will Allen. 2026. Introducing Markdown for Agents. <https://blog.cloudflare.com/markdown-for-agents/>. Accessed: 2026-04-26.
- [51] MDN contributors. 2026. Lazy loading. https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/Lazy_loading. Accessed: 2026-04-05.
- [52] MDN Web Docs contributors. 2026. Window: load event. https://developer.mozilla.org/en-US/docs/Web/API/Window/load_event. Accessed: 2026-04-20.
- [53] Media Rating Council. 2015. MRC Invalid Traffic (IVT) Detection and Filtration Guidelines Addendum. <https://www.iab.com/guidelines/mrc-invalid-traffic-ivt-detection-and-filtration-guidelines-addendum/>. Version 1.0.
- [54] Mightybytes. 2026. Ecograder. <https://ecograder.com/>. Accessed: 2026-04-16.
- [55] Mozilla. 2026. *Readability.js*. <https://github.com/mozilla/readability>. GitHub repository. Accessed: 2026-03-12.
- [56] Mozilla Support. 2024. Firefox Reader View for clutter-free web pages. <https://support.mozilla.org/en-US/kb/firefox-reader-view-clutter-free-web-pages>. Last updated August 9, 2024; accessed April 20, 2026.
- [57] Michel Nass, Emil Alégroth, Robert Feldt, Maurizio Leotta, and Filippo Ricca. 2023. Similarity-based Web Element Localization for Robust Test Automation. *ACM Trans. Softw. Eng. Methodol.* (2023).
- [58] Bing Pan, Helene A. Hembrooke, Geri K. Gay, Laura A. Granka, Matthew K. Feusner, and Jill K. Newman. 2004. The determinants of web page viewing behavior: an eye-tracking study. In *Symposium on Eye Tracking Research & Applications*.
- [59] Shauvik Roy Choudhary, Husayn Versee, and Alessandro Orso. 2010. WEBDIFF: Automated identification of cross-browser issues in web applications. In *IEEE International Conference on Software Maintenance*.
- [60] Kimberly Ruth, Aurore Fass, Jonathan Azose, Mark Pearson, Emma Thomas, Caitlin Sadowski, and Zakir Durumeric. 2022. A world wide view of browsing the world wide web. In *ACM Internet Measurement Conference*.
- [61] Kimberly Ruth, Deepak Kumar, Brandon Wang, Luke Valenta, and Zakir Durumeric. 2022. Toppling top lists: Evaluating the accuracy of popular website lists. In *ACM Internet Measurement Conference*.
- [62] Aleks Saariinen, Muhammad Zubair Farooqi, Matti Pärssinen, and Jukka Manner. 2025. Understanding and Mitigating Webpage Data Bloat: Causes and Preventive Measures. *SIGENERGY Energy Inform. Rev.* (2025).
- [63] Laurent Devernay Satyagraha, Burak Güneli, Ines Akrap, Alexander Dawson, Mike Gifford, Tim Frick, Rafael Bonalume Lebre, Lucia Harcegova, and Caleb Queern. 2024. *Sustainability*. HTTP Archive, Chapter 15. doi:10.5281/zenodo.14065377
- [64] Dave Smart, Jamie Indigo, Ines Akrap, Burak Güneli, and Montserrat Cano. 2024. *Page Weight*. HTTP Archive, Chapter 16. doi:10.5281/zenodo.14562057
- [65] Sustainable Web Design. 2025. Estimating Digital Emissions. <https://sustainablewebdesign.org/estimating-digital-emissions/>. Accessed: 2026-04-03.
- [66] Ammar Tahir, Muhammad Tahir Munir, Shaiq Munir Malik, Zafar Ayyub Qazi, and Ihsan Ayyub Qazi. 2020. Deconstructing Google’s Web Light Service. In *Web Conference*. doi:10.1145/3366423.3380168
- [67] TechnologyChecker.io. 2026. Web Traffic Statistics Q1 2026: We Analyzed Billions of Requests - Here Are the 15 Numbers That Matter. <https://technologychecker.io/blog/web-traffic-statistics>. Published: 2026-04-10; Accessed: 2026-04-27.
- [68] Narendran Thiagarajan, Gaurav Aggarwal, Angela Nicora, Dan Boneh, and Jatinder Pal Singh. 2012. Who killed my battery: Analyzing mobile browser energy consumption. *Annual Conference on World Wide Web* (2012).
- [69] Tiberiu-Septimiu Ungurianu, Sahar Al-Sudani, and Ramzi Djemai. 2025. A Practical Framework for Sustainable Website Design and Development. In *2025 12th International Conference on Social Networks Analysis, Management and Security (SNAMS)*. 410–417. doi:10.1109/SNAMS67467.2025.11391157
- [70] Tobias Urban, Martin Degeling, Thorsten Holz, and Norbert Pohlmann. 2020. Beyond the Front Page: Measuring Third Party Dynamics in the Field. In *Proceedings of The Web Conference 2020 (Taipei, Taiwan) (WWW '20)*. Association for Computing Machinery, New York, NY, USA, 1275–1286. doi:10.1145/3366423.3380203
- [71] U.S. Environmental Protection Agency. 2025. Greenhouse Gas Emissions from a Typical Passenger Vehicle. <https://www.epa.gov/greenvehicles/greenhouse-gas-emissions-typical-passenger-vehicle>. Accessed: 2026-04-09.
- [72] Antoine Vastel, Peter Snyder, and Benjamin Livshits. 2018. Who Filters the Filters: Understanding the Growth, Usefulness and Efficiency of Crowdsourced Ad Blocking. (2018). <http://arxiv.org/abs/1810.09160>
- [73] Lucas Vogel and Thomas Springer. 2022. An In-Depth Analysis of Web Page Structure and Efficiency with Focus on Optimization Potential for Initial Page Load. In *Web Engineering*, Tommaso Di Noia, In-Young Ko, Markus Schedl, and Carmelo Ardito (Eds.). Springer International Publishing, Cham, 101–116.
- [74] Philip Walton. 2020. Web Vitals. <https://web.dev/articles/vitals#core-web-vitals>. Accessed: 2026-04-26.
- [75] web.dev contributors. 2026. Browser-level image lazy loading for the web. <https://web.dev/articles/browser-level-image-lazy-loading>. Accessed: 2026-04-05.
- [76] Wholegrain Digital. 2026. Website Carbon Calculator. <https://www.websitcarbon.com/>. Accessed: 2026-04-16.
- [77] WIRED. 2025. Cloudflare Has Blocked 416 Billion AI Bot Requests Since July 1. <https://www.wired.com/story/big-interview-event-matthew-prince-cloudflare/>. Matthew Prince reported that Cloudflare blocked 416 billion AI bot requests since July 1, 2025. Accessed: 2026-04-10.
- [78] World Wide Web Consortium (W3C). 2026. Web Sustainability Guidelines (WSG). <https://w3c.github.io/sustainableweb-wsg/>. Accessed: 2026-04-15.
- [79] Xinlei Yang, Wei Liu, Hao Lin, Zhenhua Li, Feng Qian, Xianlong Wang, Yunhao Liu, and Tianyin Xu. 2023. Visual-Aware Testing and Debugging for Web Performance Optimization. In *ACM Web Conference*.
- [80] Jingyuan Zhu, Huan Chen Sun, and Harsha V. Madhyastha. 2025. Toward Better Efficiency vs. Fidelity Tradeoffs in Web Archives. In *ACM Internet Measurement Conference*. doi:10.1145/3730567.3764507

A Ethics

We took several precautions to ensure our measurements did not disrupt the usual flow of traffic for the websites we analyzed. First, because we focus on landing pages of globally popular websites, the additional requests generated by our measurements are negligible compared to their typical traffic volumes. Second, we limit our crawling behavior by visiting webpages sequentially and at most 84 times per site, which reduces the risk of inadvertently generating denial-of-service traffic patterns. We follow the best practices set forth by Durumeric et al. [24]; we received no opt-out requests from sites.

Another concern is that our study could be misused to justify more aggressive web crawling. First, our proposal to selectively load webpages is beneficial to the content providers, who would need to process far fewer requests if webpages were visited more intentionally. In this sense, it would reduce the operational burden on origin servers; a net positive for the providers. Second, our work highlights the potential for reducing the energy consumption of each crawl. Importantly, actors who disregard environmental costs would likely crawl at the same frequency regardless; our work instead advocates for reducing the extraneous consequences of such activity.

B Grounding The Fidelity Metrics

Figure 12 shows the resulting similarity scores for a webpage and two selectively loaded counterparts. The webpages are cropped to show only above-the-fold differences. Even with no visual fidelity (Figure 12b), the webpage may still have underlying structural properties that make it feasible to be automated programmatically. Hence, we additionally capture the structural fidelity.

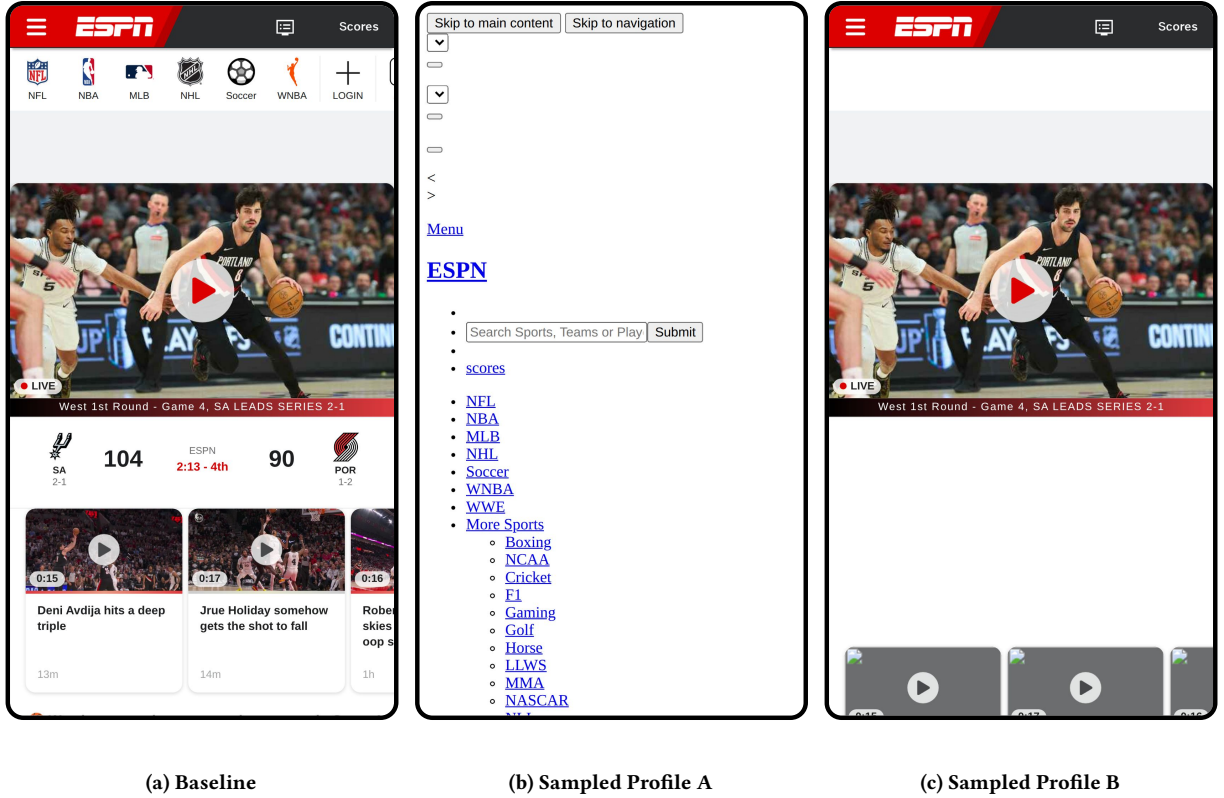


Figure 12: Example webpage renderings and their fidelity scores under different resource-loading profiles. The baseline, (a), is the reference rendering; scores for (b) and (c) are normalized to the reload of the baseline. Notably, structural and visual fidelity capture different dimensions and one may not imply the other.

C Tasks for interactivity evaluation

Category Selection. Our corpus of sampled websites consists of 4,000 websites, sampled as mentioned in Section 4.6. Of these websites, 2509 (62.7%) had at least one categorization assigned by Cloudflare’s URL Scanner API. As we rely on an underlying LLM model for our automated browser agent, the model’s built-in guardrails make it infeasible to evaluate interactivity on sites with prohibited content [3]. Thus, categories such as *Pornography*, *Adult Themes*, *CIPA*, etc., were excluded from our search. We neither advocate nor attempt to circumvent these guardrails, and instead forego interactive fidelity evaluation for these explicit risk categories. The remaining 1935 websites constitute 77.1% of the categorized websites (48.4% of the entire corpus). Of these, we define tasks for categories according to Table 4. Together, the 840 websites from the selected categories constitute 43.2% of the categorized, non-prohibited domains (21% of the entire corpus).

Category	% of websites	Tasks Generated
Entertainment	15.5	
CIPA	14.4	
CIPA Filter	14.4	
Shopping & Auctions	10.0	✓
Adult Themes	8.6	
Ecommerce	8.4	✓
News & Media	8.2	✓
Pornography	7.7	
Society & Lifestyle	6.9	
Technology	6.6	
Business & Economy	4.6	
Education	4.5	
Gambling	3.7	
Internet Communication	2.9	
Gaming	2.9	
Business	2.7	
Video Streaming	2.5	✓

Table 4: Most common categorizations among sampled URLs with at least 100 occurrences. — Categories with generated task sets are marked with ✓. Websites with multiple categorizations are counted multiple times.

■ Explicit Risk Category
■ Overly Broad Category

Task Selection. We compile tasks for the selected categories by generalizing the Mind2Web [23] and WebTailBench [7] datasets. Additionally, we include some manually generated tasks that are generally expected from websites in these categories. Two authors independently gave each task a score from 1–5, according to the task’s importance in that specific category of site. The assigned weights are the average of the two authors’ responses. The full list of tasks, categories, sources, and assigned weightages is given in Table 5.

D Additional Resource Reductions

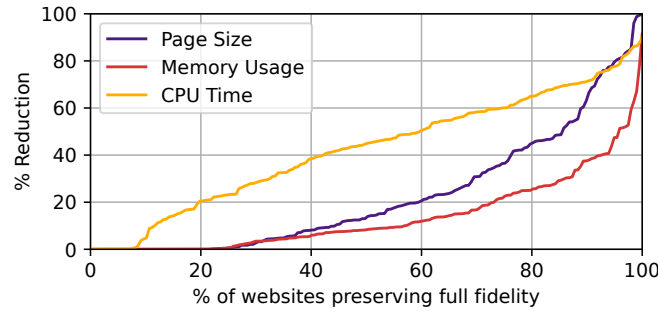


Figure 13: CDF showing resource reductions under strict fidelity. — Strict fidelity requires textual, structural, visual, and interactive fidelity to be preserved. Significant reductions can be achieved despite preserving all four dimensions of fidelity.

Task Description	Category	Attribution	Weight
Open the latest article on the homepage and summarize its main point.	News & Media	WebTailBench	5.0
Search the site for the latest news about a specific topic and open the newest relevant article.	News & Media	Mind2Web	4.5
Browse a topic page and open the top or trending headline in that topic.	News & Media	WebTailBench	4.5
Open the first or most recent article in a specific section and summarize the key takeaway.	News & Media	Mind2Web	4.0
List the newest headlines on a site for a specified topic or section.	News & Media	Mind2Web	3.5
Open the latest news release or press release and extract the media contact information.	News & Media	Mind2Web	3.0
Find the latest entertainment news about a specific movie, TV show, game, or celebrity.	News & Media	WebTailBench	2.5
Find the latest video, clip, or highlight related to a news or sports topic and play the video.	News & Media	Mind2Web	2.5
Find the most recent score, standings, roster, or player update for a specified sports team.	News & Media	WebTailBench	2.0
Find critic reviews, ratings, or audience scores for a specified movie, show, game, or article.	News & Media	Mind2Web	2.0
Leave a comment on a featured or popular article.	News & Media	Manual	2.0
Click one of the social media links associated with the article.	News & Media	Manual	2.0
Find the author of the latest article and open that author's profile or author page.	News & Media	WebTailBench	1.5
Find two other articles written by the same author as any article on the homepage.	News & Media	WebTailBench	1.5
Search for a product relevant to the site using the search feature.	Shopping & Ecommerce	WebTailBench	5.0
Add an item to cart.	Shopping & Ecommerce	WebTailBench	5.0
Remove an item from cart.	Shopping & Ecommerce	Mind2Web	5.0
Sort products by highest rating.	Shopping & Ecommerce	Mind2Web	4.5
Navigate to the checkout page and verify product details.	Shopping & Ecommerce	WebTailBench	4.5
Browse products within a category.	Shopping & Ecommerce	WebTailBench	4.0
Find the cheapest or best-value option.	Shopping & Ecommerce	WebTailBench	4.0
Navigate to the product details page of an item with multiple styles and select a different size, color, or configuration.	Shopping & Ecommerce	Manual	4.0
Sort products by lowest price.	Shopping & Ecommerce	Mind2Web	3.5
Find products on sale or within a budget of \$100.	Shopping & Ecommerce	WebTailBench	3.5
Check stock, shipping, pickup, or delivery availability.	Shopping & Ecommerce	WebTailBench	2.5
Filter products by attributes and constraints.	Shopping & Ecommerce	WebTailBench	2.0
Filter results by size or color.	Shopping & Ecommerce	Mind2Web	2.0
Compare similar products to the first product listed on the page.	Shopping & Ecommerce	WebTailBench	2.0
Filter results by brand.	Shopping & Ecommerce	Mind2Web	1.5
Search for a video with site-relevant keywords using the search bar.	Video Streaming	WebTailBench	5.0
Click on a video to open the player and stream the video.	Video Streaming	Manual	5.0
Begin playing a featured video, pause the video, wait a few seconds, and then resume the video.	Video Streaming	Manual	3.5
Begin playing a featured video and adjust the volume of the video.	Video Streaming	Manual	3.5
Begin playing a video, enter fullscreen mode, wait a few seconds, and then exit fullscreen mode.	Video Streaming	Manual	3.5
Begin playing a featured video and adjust playback speed.	Video Streaming	Mind2Web	3.0
Begin playing any video over 30 seconds, then seek to a specific, randomly chosen timestamp that is shorter than the video length.	Video Streaming	Manual	3.0
Begin playing a video, then mute the video.	Video Streaming	Manual	2.5
Filter videos by subject or keyword. Use a site-relevant keyword of choice for this.	Video Streaming	Mind2Web	2.5
Begin playing any video over 30 seconds, then fast forward the video by 10 seconds.	Video Streaming	Manual	2.5
Begin playing any video over 30 seconds, then rewind the video by 10 seconds.	Video Streaming	Manual	2.5
Add a video, TV show, or movie to the watchlist.	Video Streaming	Mind2Web	2.0
Edit an existing watchlist by adding or removing a video.	Video Streaming	Mind2Web	2.0
Filter for top videos by popularity or rating.	Video Streaming	Mind2Web	2.0

Table 5: Tasks used for interactivity evaluation – The ‘weight’ assigned to each task is the average of the weights assigned by two authors. Tasks were derived from the Mind2Web dataset [23], WebTailBench dataset [7], or manually defined.

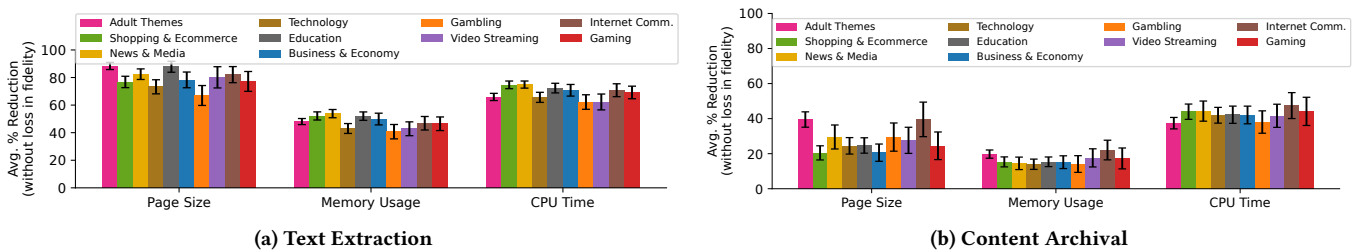


Figure 14: Category-level trade-offs for text extraction and content archival. – Reductions presented are average values. Error bars represent 95% confidence intervals.

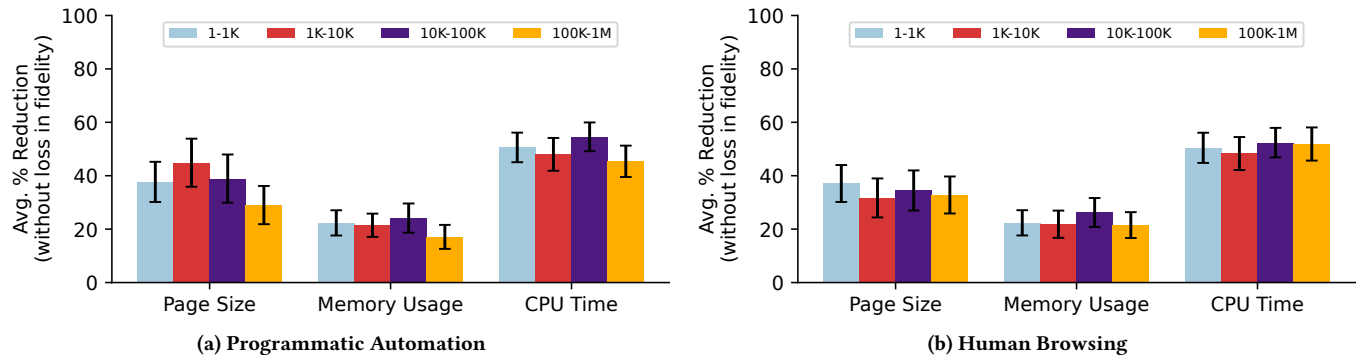


Figure 15: Popularity-level trade-offs for programmatic automation and human browsing. — Reductions presented are average values. Error bars represent 95% confidence intervals.